1. Introducción y contexto del módulo

El módulo Skills tiene dos responsabilidades:

- Describir las propiedades de cada habilidad (daño, radio de alcance, coste de energía, nombre).
- Aplicar el efecto de una habilidad sobre un Numen objetivo.

Dentro de la arquitectura del juego, el módulo se sitúa entre:

- numen.c — almacena qué skills posee cada Numen (array de Skills_id).
- game_actions.c — decide cuándo y con qué skill atacar, llamando a skill_get_radio() y skill_apply_effect().
- game_rules.c — en el futuro podrá invocar skills automáticamente en el turno de los enemigos.

Decisión de diseño importante: el módulo Skills NO gestiona objetos en el heap (osea, no creamos un objeto como skill_takle para luego acceder a sus metodos). No hay skill_create() ni skill_destroy(). Las skills son datos estáticos definidos en tiempo de compilación. El módulo es puramente funcional: recibe un Skills_id y devuelve propiedades o aplica efectos. Esto es compatible con -Wall -ansi -pedantic y Valgrind queda limpio sin esfuerzo adicional.

---

2. Arquitectura actual — Análisis del código

2.1 Estado de skills.h

El header actual define correctamente el enum y la estructura, pero NO declara ningún prototipo de función:

```
/* headers/skills.h — estado actual en el repositorio */

typedef enum {
    NO_SKILL,
    TAKLE,
    QUICK_ATTACK,
    STEAM_ERUPTION
} Skills_id;

struct _Skills {
    Skills_id  id;
    char*      name;
    int        radio;
    int        distancia;
    int        damage;
    int        coste;
};

typedef struct _Skills Skills;
```

Problemas detectados:

- No hay ningún prototipo de función declarado. Falta declarar las funciones publicas en el .h
- skill_get_radio() y skill_apply_effect() son llamadas desde game_actions.c pero no están declaradas en el header. Con -ansi esto es un error.
- El struct _Skills tiene char* name (puntero a string). Al ser la tabla estática, con literales de cadena es suficiente; no hay que alojar memoria.

2.2 Estado de skills.c

El fichero de implementación tiene versión 0 con bugs graves, además esta declarado con skill_active, que ya no se usará lo cambiaremos por skill_apply_effect()

2.3 Cómo game_actions.c depende de Skills

En game_actions_attack() ya existe la lógica completa de combate por distancia(Salva está encargada de esta parte, cualquier duda, preguntale). El fragmento que espera que Skills funcione es:

```
/* Fragmento de game_actions.c — ya escrito, necesita Skills */

skill = numen_get_skill_by_index(num, skill_indx); /*FALTA IMPLEMENTAR!!*/
if (skill == NO_SKILL) { /* error */ return; }

radio = skill_get_radio(skill);            /* FALTA IMPLEMENTAR */

/* ... bucle por numens enemigos en el espacio ... */
distance = sqrt(pow(dx,2) + pow(dy,2));

if (distance <= radio) {
    skill_apply_effect(skill, num, enemy_num); /* FALTA IMPLEMENTAR */ ESTA ES
FUNDAMENTAL QUE ESTÉ DEFINIDA
}
```

Por tanto, las tres funciones mínimas para que el combate funcione son **skill_get_radio(), numen_get_skill_by_index()** y **skill_apply_effect()**. Son el camino crítico.

---

3. Tabla de estado completo

Skills_id (enum)	Hecho	—
struct _Skills (definición)	Hecho	—
typedef Skills	Hecho	—
Prototipos en skills.h	**FALTA**	*Crítico*
Tabla estática de datos	**FALTA**	*Crítico*
skill_get_radio()	**FALTA**	*Crítico*
skill_apply_effect()	**FALTA**	*Crítico*
skill_get_damage()	**FALTA**	*Medio*
skill_get_coste()	**FALTA**	*Medio*
skill_get_name()	**FALTA**	*Bajo*
skill_active() — corrección	***QUITAR***	*Crítico*

`skill_effect_takle()`	***ADAPTAR***	*Crítico*
`skill_effect_quick_attack()`	**FALTA**	*Medio*
`skill_effect_steam_eruption()`	**FALTA**	*Medio*
`skills_test.c`	**FALTA**	*Medio*

---

4. Lo que falta por hacer — Paso a paso

Paso 0 — Resolver la confusión de tipos (antes de todo)

Este es el primer problema que hay que solucionar porque afecta a todos los demás.

El problema concreto:

```
/* game_actions.c maneja: */
Skills_id skill;          /* el ENUM — es un int */

/* skills.c declara: */
void skill_active(Numen*, Numen*, Skills id_skill);
                                  ^^^^^^
                          esto es el STRUCT, no el enum
```

La solución: todas las funciones públicas de skills.c deben recibir Skills_id (el enum), NO Skills (el struct). El struct es un detalle interno del módulo.

Paso 1 — Completar skills.h con los prototipos

Nota sobre el forward declaration: numen.h incluye skills.h, así que skills.h NO puede incluir numen.h o habría un ciclo de dependencias. La línea 'typedef struct _Numen Numen;' le dice al compilador que Numen existe como tipo sin necesitar el struct completo. El linker lo resolverá cuando se compile skills.c, que sí incluye numen.h para poder llamar a sus funciones.

Paso 2 — Tabla de datos estática (el corazón del módulo)

En lugar de crear Skills dinámicamente con malloc, se define un array estático const en skills.c indexado por Skills_id. Esto funciona porque Skills_id es un enum que empieza en 0, así que los valores del enum son directamente los índices del array. Esto lo he terminado definiendo en parte, pero requiero que se hagan sus dependencias

```
/* src/skills.c — tabla privada al principio del fichero */

#define NUM_SKILLS 4

static const Skills skill_table[NUM_SKILLS] = {
    /* [NO_SKILL]       */ { NO_SKILL,      "Ninguna",        0, 0,  0,  0 },
    /* [TAKLE]          */ { TAKLE,          "Tackle",         3, 1,  5,  2 },
    /* [QUICK_ATTACK]   */ { QUICK_ATTACK,   "Quick Attack",   2, 1,  3,  1 },
    /* [STEAM_ERUPTION] */ { STEAM_ERUPTION, "Steam Eruption", 5, 2, 10,  4 },
};
/* Campos en orden: id, name, radio, distancia, damage, coste */
/* Los valores numéricos los define el equipo según balance de juego */
```

Con esta tabla, implementar cualquier getter es una sola línea. No hay malloc, no hay free, no hay posibilidad de leak.

Paso 3 — skill_get_radio() y los demás getters

Todos los getters siguen el mismo patrón:

```
/* Pseudocódigo de todos los getters */

FUNCIÓN skill_get_radio(Skills_id id):
    SI id < 0 O id >= NUM_SKILLS:
        DEVOLVER 0          /* valor seguro: radio 0 cancela el ataque */
    DEVOLVER skill_table[id].radio

FUNCIÓN skill_get_damage(Skills_id id):
    SI id < 0 O id >= NUM_SKILLS: DEVOLVER 0
    DEVOLVER skill_table[id].damage

FUNCIÓN skill_get_coste(Skills_id id):
    SI id < 0 O id >= NUM_SKILLS: DEVOLVER 0
    DEVOLVER skill_table[id].coste

FUNCIÓN skill_get_name(Skills_id id):
    SI id < 0 O id >= NUM_SKILLS: DEVOLVER "DESCONOCIDA"
    DEVOLVER skill_table[id].name

Por eso es fundamental la tabla, de esa forma podemos acceder sin muchos problemas a a
cada campo, y como es no va a cambiar dará igual cuantos espacios hayq ue guardar a
name o algo, solo lo devolvemos rápidamente
```

Nota: skill_get_radio() devuelve int, no Status, porque game_actions.c lo usa directamente como número (radio = skill_get_radio(skill)). Devolver 0 en caso de error tiene semántica clara: radio 0 significa que el ataque nunca alcanza a nadie.

Paso 4 — skill_apply_effect() — La función clave

Esta es la función más importante del módulo. Recibe la identidad de la skill, quién la lanza (sender) y quién la recibe (receiver). Su responsabilidad es exactamente una: validar, cobrar energía, y despachar al efecto concreto de cada skill.

```
/* Pseudocódigo de skill_apply_effect */

FUNCIÓN skill_apply_effect(Skills_id id, Numen* sender, Numen* receiver) →
Status:

    SI sender == NULL O receiver == NULL:
        DEVOLVER ERROR

    SI id == NO_SKILL O id < 0 O id >= NUM_SKILLS:
        DEVOLVER ERROR

    /* ¿Tiene el sender suficiente energía? */
    SI numen_get_energy(sender) < skill_table[id].coste:
        DEVOLVER ERROR

    /* Descontar el coste de energía */
    numen_set_energy(sender, numen_get_energy(sender) - skill_table[id].coste)
```

```
    /* Despachar al efecto concreto */
    SEGÚN id:
        CASO TAKLE:          DEVOLVER skill_effect_takle(sender, receiver)
        CASO QUICK_ATTACK:   DEVOLVER skill_effect_quick_attack(sender,
receiver)
        CASO STEAM_ERUPTION: DEVOLVER skill_effect_steam_eruption(sender,
receiver)
        DEFAULT:             DEVOLVER ERROR
```

–Cada efecto concreto deberá de tener un imaplementación—

Estructura de dos capas: skills apply effect() es pública (declarada en skills.h) y hace validación + coste + dispatch. Las funciones **skill_effect_takle(), skill_effect_quick_attack(), etc. son privadas** (static, solo en skills.c) y hacen el efecto concreto. En C, 'static' en una función de fichero significa que el linker no la exporta — equivale a un método privado — Por eso lo usaremos en todo nuestro codigo para ahorrarnos cietar declaraciones y mantender cierta coherencia

Paso 5 — Implementar el efecto concreto de cada skill

El patrón general de cualquier skill de daño directo:

```
/* Pseudocódigo de skill_effect_takle */

FUNCIÓN PRIVADA skill_effect_takle(Numen* sender, Numen* receiver) → Status:

    SI sender == NULL O receiver == NULL: DEVOLVER ERROR

/*skill_type_advantage() devolverá un número aleatorio entre 0 y 0.5 si el
sender tiene ventaja de tipo- es fuego contra planta- o un número negativo
entre 0 y menos 0.5 si tiene desventaja -Agua contra planta-*/
    Advantage = skill_type_advantage(sender, receiver) ;

/* Daño total = ataque del sender + (daño base de la skill * ventaja de tipo)
*/
    daño_total = numen_get_attack(sender) + skill_table[TAKLE].damage *
Advantage

    /* La vida nunca puede ser negativa (regla del proyecto) */
    nueva_vida = numen_get_health(receiver) - daño_total
    SI nueva_vida < 0: nueva_vida = 0

    numen_set_health(receiver, nueva_vida)
    DEVOLVER OK


/* QUICK_ATTACK: igual que TAKLE pero con damage base distinto */

FUNCIÓN PRIVADA skill_effect_quick_attack(Numen* sender, Numen* receiver) →
Status:

    Advantage = skill_type_advantage(sender, receiver) ;
daño_total = numen_get_attack(sender) + skill_table[QUICK_ATTACK].damage * Advantage;

    nueva_vida = MAX(0, numen_get_health(receiver) - daño_total)
    numen_set_health(receiver, nueva_vida)
    DEVOLVER OK
```

```
/* STEAM_ERUPTION: daño alto a un objetivo, radio grande */
/* El efecto de área (alcanzar a varios) lo gestiona el bucle de game_actions.c
*/
/* Esta función solo aplica daño a UN objetivo a la vez */

FUNCIÓN PRIVADA skill_effect_steam_eruption(Numen* sender, Numen* receiver) →
Status:
    Advantage = skill_type_advantage(sender, receiver) ;
daño_total = numen_get_attack(sender) + skill_table[STEAM_ERUPTION].damage * Advantage
    nueva_vida = MAX(0, numen_get_health(receiver) - daño_total)
    numen_set_health(receiver, nueva_vida)
    DEVOLVER OK
```

Paso 6 — Corregir skill_active()

La función actual tiene bugs de compilación. La solución recomendada es renombrarla directamente como skill_apply_effect() con la firma correcta. Si se quiere mantener skill_active() como nombre alternativo:

```
/* Opción recomendada: renombrar directamente */
Status skill_apply_effect(Skills_id id, Numen *sender, Numen *receiver)
{
    /* validaciones + coste energía + dispatch (ver Paso 4) */
}

/* Eliminar: */
/*   - el 'void' que causaba 'return ERROR' imposible          */
/*   - skill_active_takle() sin tipo de retorno                */
/* Renombrar a skill_effect_takle() con 'static Status'        */
```

Paso 7 — Pruebas unitarias (skills_test.c)

Las pruebas de Skills no están en P1 (que cubre Space, Object, Player, Set, Character, Link, Inventory), pero la prueba de integración P3 las necesita implícitamente para que 'attack' funcione.

```
/* Pseudocódigo de las pruebas mínimas */

TEST test1_skill_get_radio_valido:
    COMPROBAR skill_get_radio(TAKLE) > 0

TEST test2_skill_get_radio_no_skill:
    COMPROBAR skill_get_radio(NO_SKILL) == 0

TEST test3_skill_get_radio_invalido:
    COMPROBAR skill_get_radio(-1) == 0

TEST test1_skill_apply_effect_takle:
    sender   = numen_create()  → vida=10, ataque=3, energía=5
    receiver = numen_create()  → vida=20
    COMPROBAR skill_apply_effect(TAKLE, sender, receiver) == OK
    COMPROBAR numen_get_health(receiver) < 20   /* la vida bajó */
    numen_destroy(sender); numen_destroy(receiver)

TEST test2_skill_apply_effect_sin_energia:
    sender = numen_create()  → energía=0
    COMPROBAR skill_apply_effect(TAKLE, sender, receiver) == ERROR
    numen_destroy(sender); numen_destroy(receiver)

TEST test3_skill_apply_effect_null:
```

```
    COMPROBAR skill_apply_effect(TAKLE, NULL, NULL) == ERROR

TEST test4_vida_no_negativa:
    sender   = numen_create()  → ataque=999, energía=99
    receiver = numen_create()  → vida=1
    skill_apply_effect(TAKLE, sender, receiver)
    COMPROBAR numen_get_health(receiver) == 0   /* nunca negativa */
    numen_destroy(sender); numen_destroy(receiver)
```

---

5. Integración con game_actions.c

Con el módulo completo, el flujo de combate en game_actions_attack() quedará así:

```
/* Pseudocódigo del flujo completo de combate */

1. Obtener jugador activo → player
2. Obtener Numen activo → num
3. Obtener índice de skill del comando → skill_indx
4. skill = numen_get_skill_by_index(num, skill_indx)
5. SI skill == NO_SKILL → ERROR, salir

6. radio = skill_get_radio(skill)              ← Paso 3 implementado

7. PARA cada numen_enemigo en el espacio:
       dx = pos_x_activo - pos_x_enemigo
       dy = pos_y_activo - pos_y_enemigo
       distancia = sqrt(dx*dx + dy*dy)
       SI distancia <= radio:
           skill_apply_effect(skill, num, numen_enemigo)  ← Paso 4 implementado

8. game_set_last_cmd_status(game, OK)
```

Punto de atención — energía vs. turnos: skill_apply_effect() descuenta la energía **EN CADA APLICACIÓN**. Si Steam Eruption alcanza a 3 enemigos en rango, se descuenta energía 3 veces. Alternativa: descontar energía UNA SOLA VEZ en game_actions.c antes del bucle y que skill_apply_effect() no la descuente. Por ahora implementar en el codigo de skill, pero deja un comentario arriba de ella que diga –/*HAY QUE DEBATIR ESTO*/– para acordarme de ello, lo tengo que discutir con Salva, aunque creo que lo mejor es que esté en game_actions_attack, pero luego lo veremos entre todos.

---

6. Orden de implementación

Este es el orden que garantiza que cada paso compila y supera la corrección automática:

1. Corregir tipos en skills.h — prototipos con Skills_id (no Skills). Resultado: compila sin warnings con -Wall -ansi -pedantic.
2. Añadir forward declaration de Numen en skills.h y resolver el ciclo de dependencias.
3. Crear skill_table[] estática en skills.c con los 4 skills. Resultado: valores accesibles internamente, sin malloc/free.

4. Implementar skill_get_radio(), skill_get_damage(), skill_get_coste(). Resultado: game_actions_attack() puede calcular rango — el combate por distancia funciona.
5. Implementar skill_apply_effect() con dispatch por switch. Resultado: el ataque aplica daño y consume energía; vida nunca < 0.
6. Implementar skill_effect_takle(), skill_effect_quick_attack(), skill_effect_steam_eruption(). Resultado: cada skill tiene comportamiento distinto.
7. Escribir skills_test.c con al menos 8 pruebas. Resultado: el script P2 la ejecuta; Valgrind: 0 leaks.

Otra cosa, hay un aparte del eneucniado que nos exogen que le quitemos aleatoriedad en caso de que se hagan test, de eso ya me he encargado en game_loop, asi que puedes usar rand() sin ningún probelema porque desde un inicio el que vaya atestar ya habra tenido que decirnos si quiere una semilla única o una dada por el tiempo del sistema.